Loan Approval Prediction

A Random Forest Project

Name:	Karthik Somaraju
Submission Date:	11.10.2026
Project Type:	Random Forest (Classification)
Dataset:	Loan Approval Dataset
Dataset Source:	Kaggle
Kaggle Link:	https://www.kaggle.com/datasets/rohitgrewal/loan-approval-dataset
Code Notebook (Colab):	https://colab.research.google.com/drive/1ELqZ_-9k_TkqOicY66x_GhnGjNeUZ05F?usp=sharing

2. Objective

Build a Random Forest classification model to predict whether a loan application is Approved or Rejected (loan_status: 0 = Approved, 1 = Rejected) based on the applicant details - number of dependents, education, self employment, annual income, loan amount, loan term, CIBIL score, and residential, commercial, luxury and bank asset values. The goal is to understand which features most strongly influence the loan decision and to evaluate how well the model classifies unseen applications (measured via confusion matrix, precision, recall, F1-score, ROC-AUC and feature importance).

Workflow:

Load the Kaggle CSV > inspect the dataset > clean column names and text values > exploratory analysis > handle null values and drop loan_id > label encoding > correlation and outlier check > 80:20 train test split > SMOTE on the training data > baseline Random Forest > GridSearchCV tuning > evaluate the model > feature importance.

3. Dataset Overview

The dataset is the Loan Approval Dataset from Kaggle. It contains 4269 rows and 13 columns of applicant and loan details, with a target column (loan_status) showing whether the loan was Approved or Rejected.

Property	Value
Rows	4,269
Columns	13 (12 after dropping loan_id)
Target variable	loan_status (text; Approved / Rejected, encoded as Approved = 0, Rejected = 1)
Integer features (int64)	loan_id, no_of_dependents, income_annum, loan_amount, loan_term, cibil_score, residential_assets_value, commercial_assets_value, luxury_assets_value, bank_asset_value
Categorical features	education (Graduate / Not Graduate), self_employed (Yes / No)
Float features	None

Sample records (first 5 rows):

loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	loan_status
1	2	Graduate	No	9600000	29900000	12	778	Approved
2	0	Not Graduate	Yes	4100000	12200000	8	417	Rejected
3	3	Graduate	No	9100000	29700000	20	506	Rejected
4	3	Graduate	No	8200000	30700000	8	467	Rejected
5	5	Not Graduate	Yes	9800000	24200000	20	382	Rejected

Note: the four asset value columns are hidden in this preview to fit the page width.

4. Data Quality and Cleaning

Check	Before Cleaning	After Cleaning
Rows	4,269	4,269
Duplicate rows	0	0
Missing values (all columns)	0	0
Column names with extra leading spaces	11 of 13	0
Text columns with extra spaces in values (education, self_employed, loan_status)	3 of 3	0
Total columns	13	12 (loan_id dropped)

Outlier check:

The boxplots of income_annum, loan_amount and cibil_score (see Section 6) show no points beyond the whiskers. No rows were removed, so all 4269 rows were retained (3415 for training and 854 for testing). Random Forest is also not much affected by extreme values because trees split on thresholds.

Cleaning decisions:

- No duplicate rows were found, so no rows were dropped for duplication.
- No missing values were found. A median fill step on cibil_score was kept only as a safety step and it changed nothing.
- The column names and the text values had extra spaces (for example " loan_status" and " Approved"), so they were stripped. Without this the column names cannot be used in the code.
- loan_id is only a serial number and carries no information about the applicant, so it was dropped.
- Label Encoding was used for the text columns: education (Graduate = 0, Not Graduate = 1), self_employed (No = 0, Yes = 1) and loan_status (Approved = 0, Rejected = 1).
- **Feature scaling was skipped.** Random Forest is made of decision trees that only compare a feature with a threshold (like cibil_score > 550), so they do not use distances and the size of the numbers does not matter.
- The data was split 80:20 first and SMOTE was applied only on the training data, so the test set stays untouched.

5. Descriptive Analysis

Summary for the target variable (loan_status):

Category	Meaning	Count	% of Data
0	Approved	2,656	62.22%
1	Rejected	1,613	37.78%

About 62% of the applications are Approved and about 38% are Rejected, so there is a small class imbalance (not as large as in fraud data). To be safe, SMOTE was applied on the training data so the model sees both classes equally.

Training data	Approved (0)	Rejected (1)	Total rows
Before SMOTE	2,120	1,295	3,415
After SMOTE	2,120	2,120	4,240

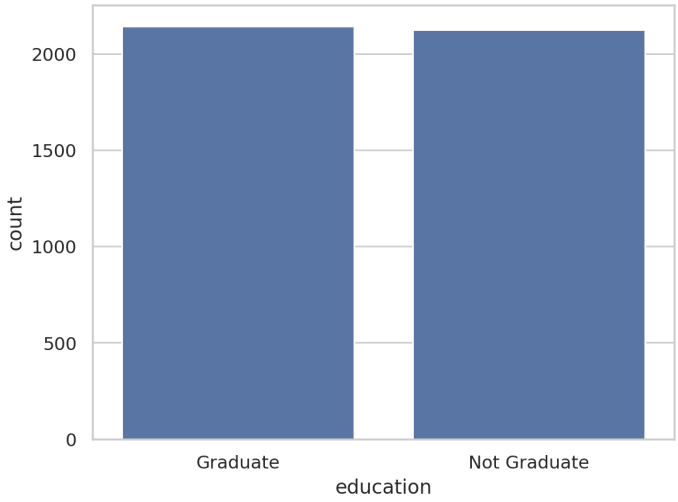
6. Visual Analysis

Chart 1: Distribution of Loan Status (Univariate)



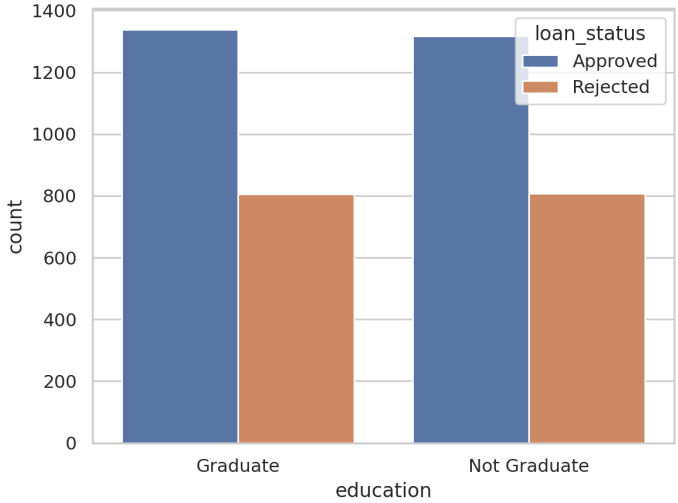
Observation: About 62% of the applications are Approved (0) and about 38% are Rejected (1), showing the small class imbalance.

Chart 2: Distribution of Education (Univariate)



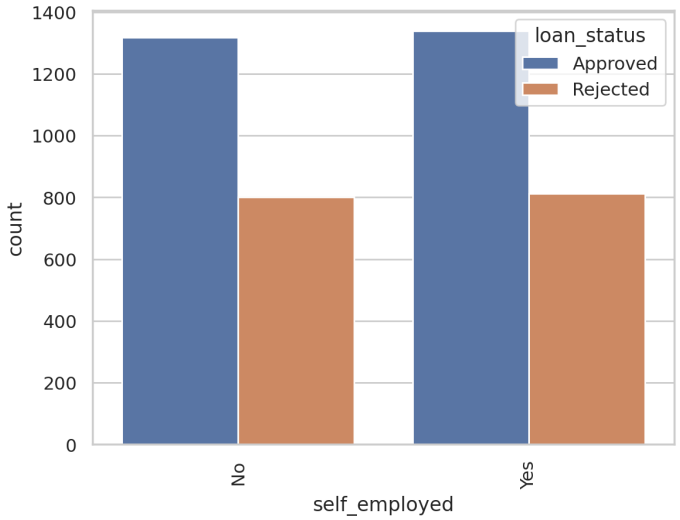
Observation: The applicants are almost equally split between Graduate (2,144) and Not Graduate (2,125).

Chart 3: Education vs Loan Status (Bivariate)



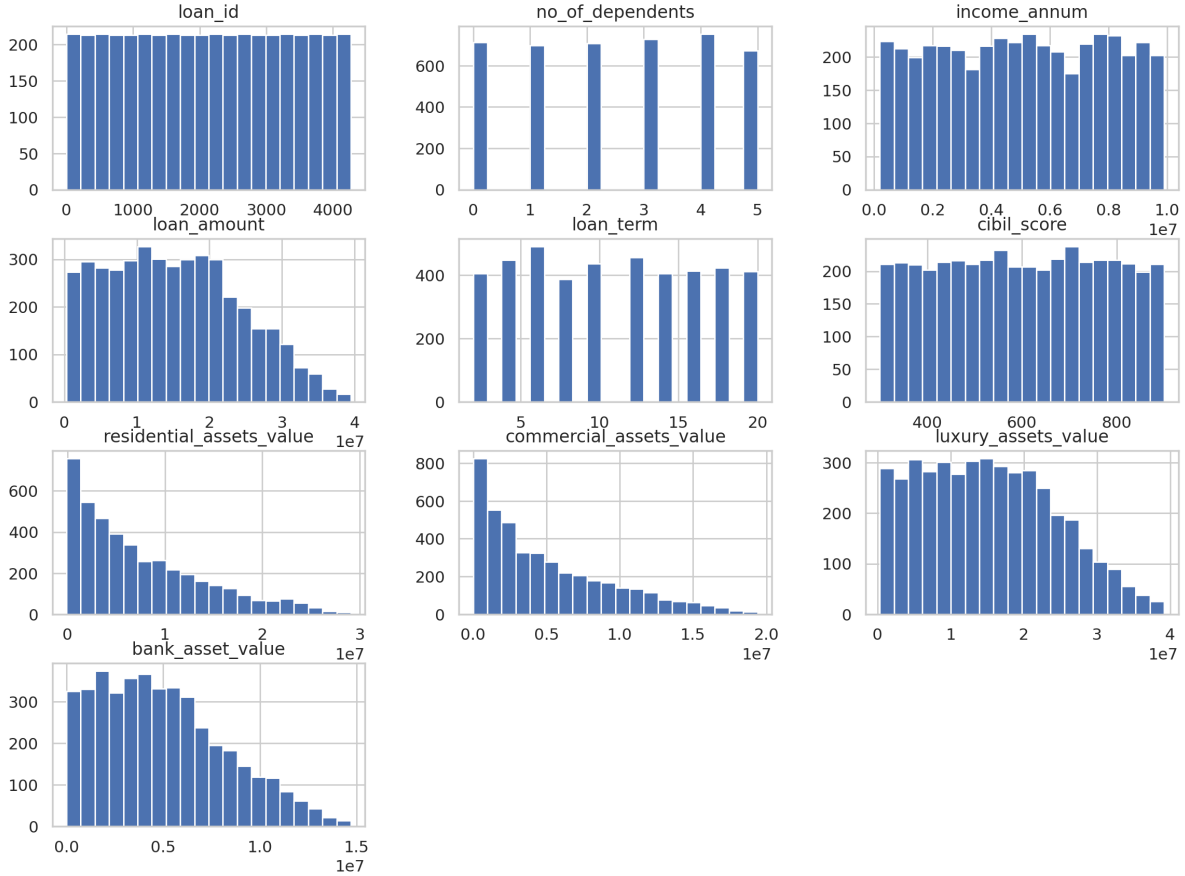
Observation: The approval rate is almost the same for both groups (about 62.5% for Graduate and 62.0% for Not Graduate), so education alone does not decide the loan outcome.

Chart 4: Self Employed vs Loan Status (Bivariate)



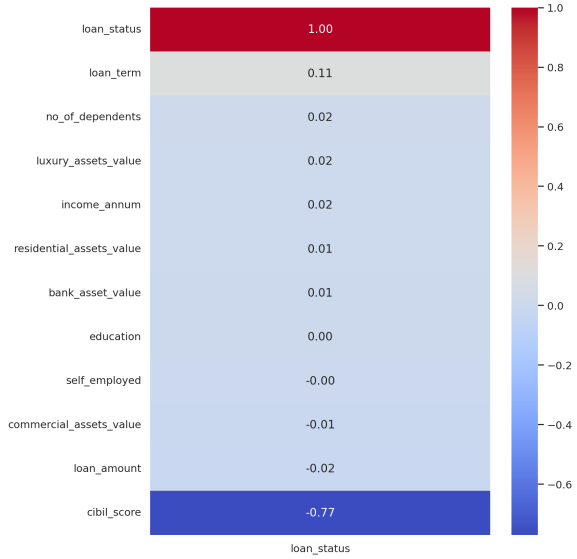
Observation: Self employed and not self employed applicants both have the same approval rate (about 62.2%), so this feature also does not separate the two classes by itself.

Chart 5: Distribution of Numeric Columns



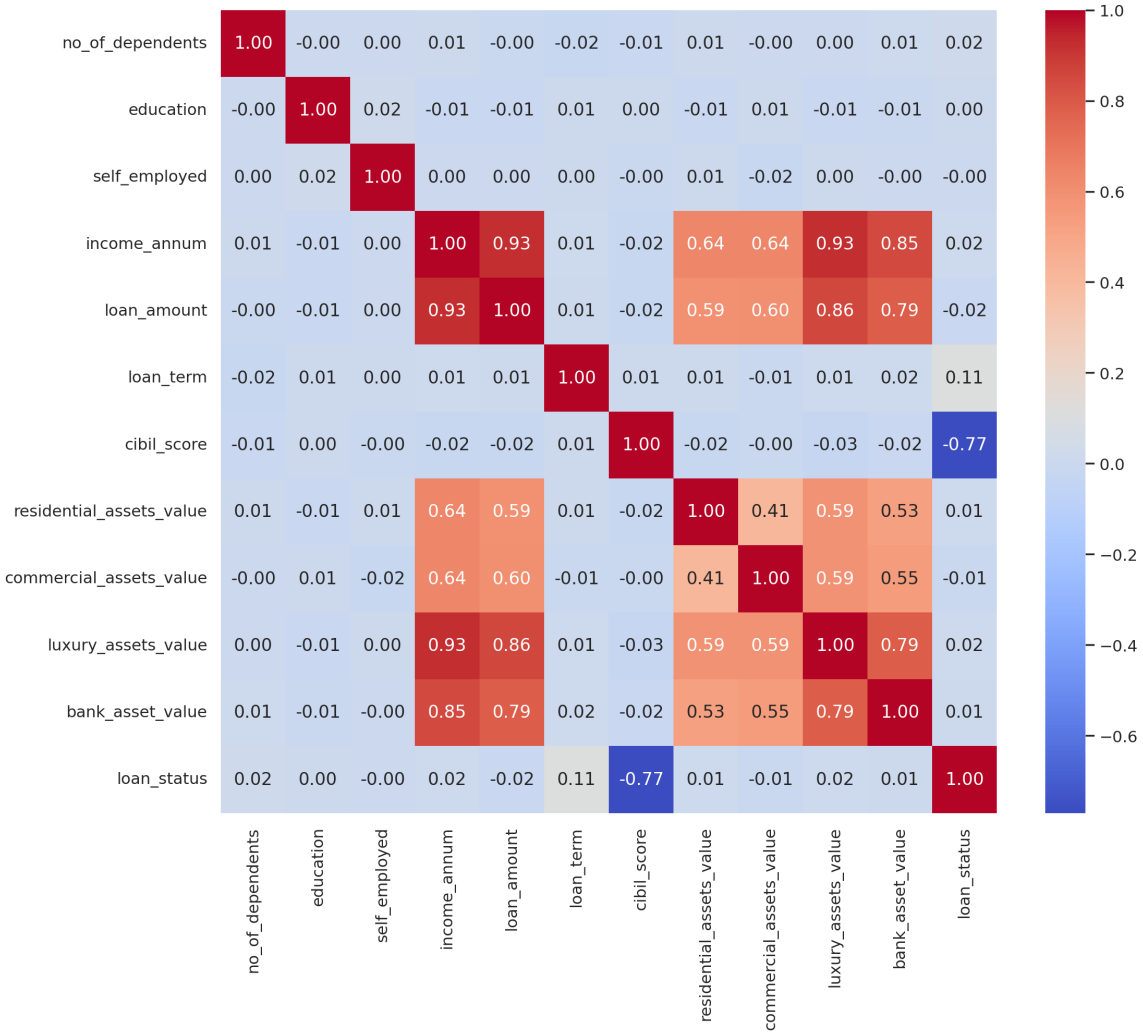
Observation: loan_id is just a serial number (flat). income_annum, cibil_score, no_of_dependents and loan_term are spread fairly evenly. residential_assets_value and commercial_assets_value are strongly right-skewed (skewness about 0.97), with most values at the lower end and a long tail. loan_amount, luxury_assets_value and bank_asset_value are only slightly skewed.

Chart 6: Correlation of Features with Loan Status



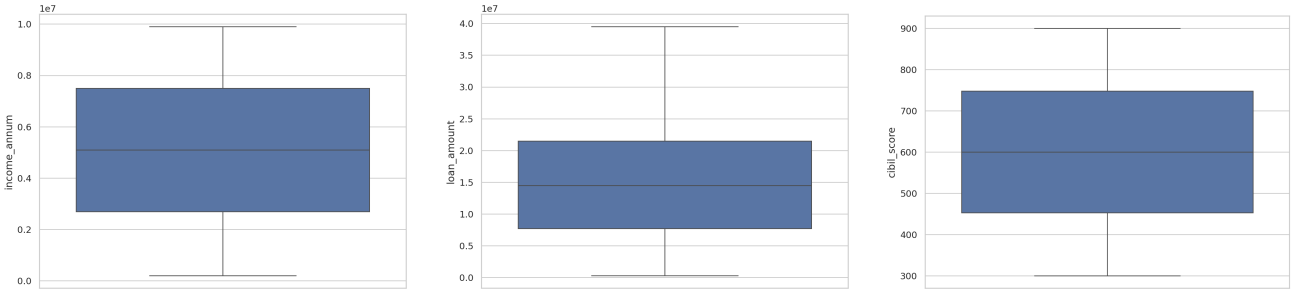
Observation: cibil_score has a strong negative correlation with loan_status (-0.77). Since Rejected = 1, a higher CIBIL score means a lower chance of rejection. loan_term has a small positive correlation (0.11) and all the other features are very close to zero.

Chart 7: Correlation Matrix



Observation: cibil_score is the only feature strongly related to loan_status (-0.77). Among the input features, income_annum is highly correlated with loan_amount (0.93) and luxury_assets_value (0.93), and loan_amount with luxury_assets_value (0.86). Random Forest can handle these related features, so none were removed.

Chart 8: Outlier Detection Boxplots



Observation: income_annum, loan_amount and cibil_score (left to right) show no points beyond the whiskers, so no outlier treatment was needed.

7. Model Building and Evaluation

The data was split 80:20 into training and test sets (random_state=42), giving training shape of (3415, 11) and test shape of (854, 11). SMOTE (random_state=42) was applied on the training part only, which made the training set (4240, 11) with 2,120 rows in each class. No feature scaling was used because Random Forest is not distance based.

7.1 Baseline Random Forest

First a Random Forest with default settings was trained. It gets a train accuracy of 1.0000 but a test accuracy of 0.9719, which means the model memorises the training data a little (overfitting). To reduce this, the hyperparameters were tuned.

Baseline model	Train accuracy	Test accuracy	Test precision (Rejected)	Test recall (Rejected)
Random Forest (default)	1.0000	0.9719	0.9712	0.9528

7.2 Hyperparameter Tuning and Justification

GridSearchCV (3-fold cross validation, scoring = F1, random_state = 42) was run on the SMOTE training data with this grid: n_estimators = [100, 200], max_depth = [5, 10], min_samples_split = [2, 5], min_samples_leaf = [1, 2]. F1 was used for scoring because it takes both precision and recall into account.

Best parameters: n_estimators = 200, max_depth = 10, min_samples_split = 2, min_samples_leaf = 1, with a cross validation F1 of 0.9741.

Top 5 parameter combinations:

n_estimators	max_depth	min_samples_split	min_samples_leaf	CV F1-score
200	10	2	1	0.9741
100	10	2	2	0.9740
100	10	5	1	0.9740
200	10	5	1	0.9735
100	10	2	1	0.9734

Why these values:

- **n_estimators (number of trees) = 200:** more trees make the prediction more stable because the average of many trees has less variance. The average CV F1 was 0.9704 for 100 trees and 0.9706 for 200 trees, so the gain is tiny. 200 is enough, and more trees would only make training slower.
- **max_depth (maximum depth of each tree) = 10:** trees with no depth limit grow until they memorise the training data (the baseline had a train accuracy of 1.0). Limiting the depth keeps each tree simpler so it generalises better. The average CV F1 was 0.9676 for depth 5 and 0.9734 for depth 10, so depth 5 was a bit too shallow and depth 10 was better.
- **min_samples_split and min_samples_leaf:** these made almost no difference, and the default values (2 and 1) gave the top score, so they were kept.

7.3 Final Model Results (Test Set)

The final model (best parameters above) has a train accuracy of 0.9998 and a test accuracy of 0.9707. The metrics below are for the Rejected class (positive class = 1).

Metric	Value
Accuracy	0.9707
Precision	0.9681
Recall	0.9528
F1-score	0.9604
ROC-AUC	0.9979

The model correctly classifies about 97.1% of the test applications. A precision of 96.8% means that when the model says a loan will be Rejected, it is right about 97% of the time (exactness). A recall of 95.3% means it finds about 95% of all the loans that were actually Rejected (completeness).

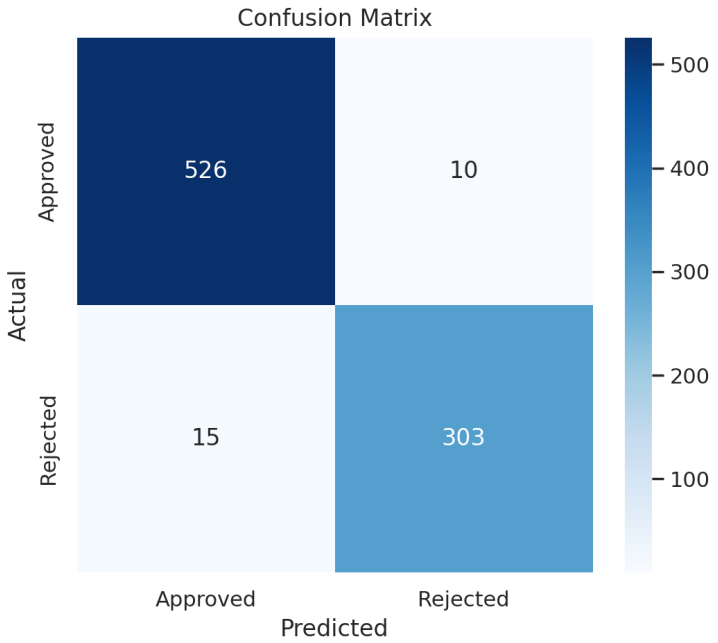
Precision and recall trade-off: the two pull in opposite directions. If the bank makes the model reject more loans to catch every risky application (higher recall), more good applicants get rejected by mistake (lower precision). If it makes the model stricter about saying Rejected (higher precision), more risky loans slip through as Approved (lower recall). For a bank the missed rejections are the more costly mistake, so recall should be watched closely. Here recall (0.953) is a little lower than precision (0.968), which means the model misses a few risky loans rather than wrongly rejecting many good ones.

F1-score: the harmonic mean of precision and recall, which is 0.960. It is high only when both are high, and it is useful here because the classes are not perfectly balanced.

Classification report:

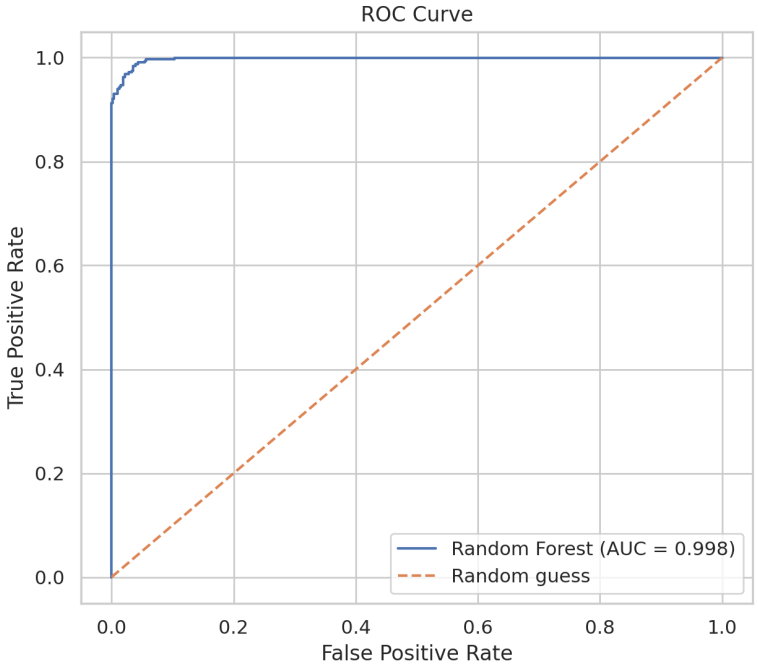
Class	Precision	Recall	F1-score	Support
0 (Approved)	0.97	0.98	0.98	536
1 (Rejected)	0.97	0.95	0.96	318
Accuracy			0.97	854
Macro avg	0.97	0.97	0.97	854
Weighted avg	0.97	0.97	0.97	854

Confusion matrix:



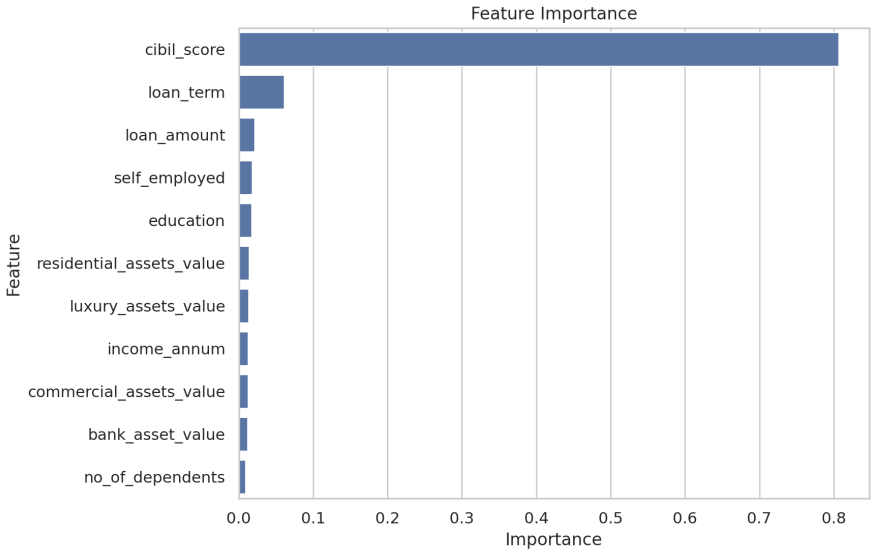
The confusion matrix shows 526 true negatives (Approved predicted Approved), 10 false positives (Approved predicted Rejected), 15 false negatives (Rejected predicted Approved), and 303 true positives (Rejected predicted Rejected).

ROC curve and AUC:



The ROC curve plots the true positive rate against the false positive rate at different thresholds. The curve rises almost straight to the top left corner and stays far above the dashed random guess line. The AUC score is 0.9979 (a random guess gives 0.5), so the model separates Approved and Rejected applications very well.

Feature importance:



Feature	Importance
cibil_score	0.807169
loan_term	0.061488
loan_amount	0.021701
self_employed	0.018339
education	0.017580
residential_assets_value	0.014008
luxury_assets_value	0.013211
income_annum	0.012866
commercial_assets_value	0.012489
bank_asset_value	0.011698
no_of_dependents	0.009452

cibil_score is by far the most important feature (about 81% of the total importance), followed by loan_term (about 6%). All the other features are below 3% each. This matches the correlation analysis, where cibil_score was the only strongly related feature. loan_term also matters because shorter terms (2 and 4 years) have a lower rejection rate (about 18-22%) than longer terms (about 39-47%).

8. Key Findings

- cibil_score is the strongest driver of the loan decision - it has the highest correlation with loan_status (-0.77) and the largest feature importance (about 81%).
- loan_term is the second most useful feature, while education, self_employed, income and the asset values have only a small influence in this model.
- The classes are slightly imbalanced (about 62% vs 38%), so SMOTE was applied on the training data only.

- The tuned Random Forest reaches an accuracy of 97.1%, precision of 96.8%, recall of 95.3%, F1-score of 0.960 and ROC-AUC of 0.998 on the test set.
- Of the 318 actual Rejected applications in the test set, 303 were correctly identified and 15 were missed (predicted as Approved).

Limitations:

- The results are for learning purposes and should not be used for real lending decisions.
- Recall is 95.3%: 15 risky (Rejected) applications in the test set were predicted as Approved, which is the costly mistake for a bank.
- Precision is 96.8%: 10 good applications in the test set were predicted as Rejected (false positives).
- The model depends mostly on one feature (cibil_score), so wrong or missing CIBIL values would hurt the predictions.
- Metrics come from a single fixed test split of 854 rows, so they may change a little with a different split. The default model is also trained without a fixed random_state, so its numbers can vary slightly between runs.
- Feature importance shows how much the model used a feature, not cause and effect. Random Forest is also harder to explain than a single decision tree.

9. Conclusion

This project applied a full Random Forest workflow - data inspection, cleaning, exploratory analysis, label encoding, train test split, SMOTE, hyperparameter tuning, evaluation and feature importance - to the Loan Approval dataset. The tuned model (n_estimators = 200, max_depth = 10) achieved an accuracy of about 97.1%, precision of 96.8%, recall of 95.3%, F1-score of 96.0% and ROC-AUC of 0.998. cibil_score was by far the most influential feature, followed by loan_term.

Strengths: very high accuracy and AUC, no feature scaling needed, handles the related income and asset columns well, and limiting max_depth reduced the overfitting of the default model.

Weaknesses: a few risky loans are still approved (recall is not 100%), the model leans heavily on one feature, and a small gap remains between train and test accuracy.

Possible improvements: try Gradient Boosting, XGBoost or LightGBM and compare F1 and AUC; tune the hyperparameters further (RandomizedSearchCV or a bigger grid) or change the decision threshold to raise recall; create new features such as loan amount to income ratio or total assets; and use cross validation on the final model to confirm that the scores are stable.